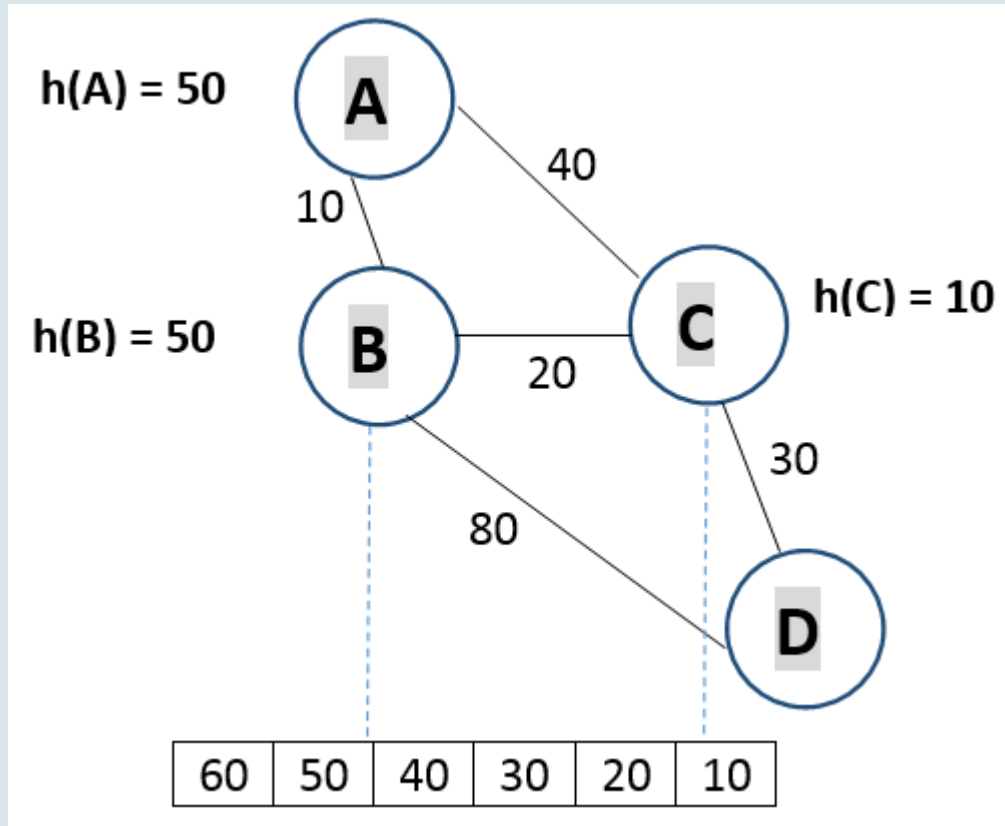


Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality



An **admissible** heuristic will ensure optimality when we use **tree search**.

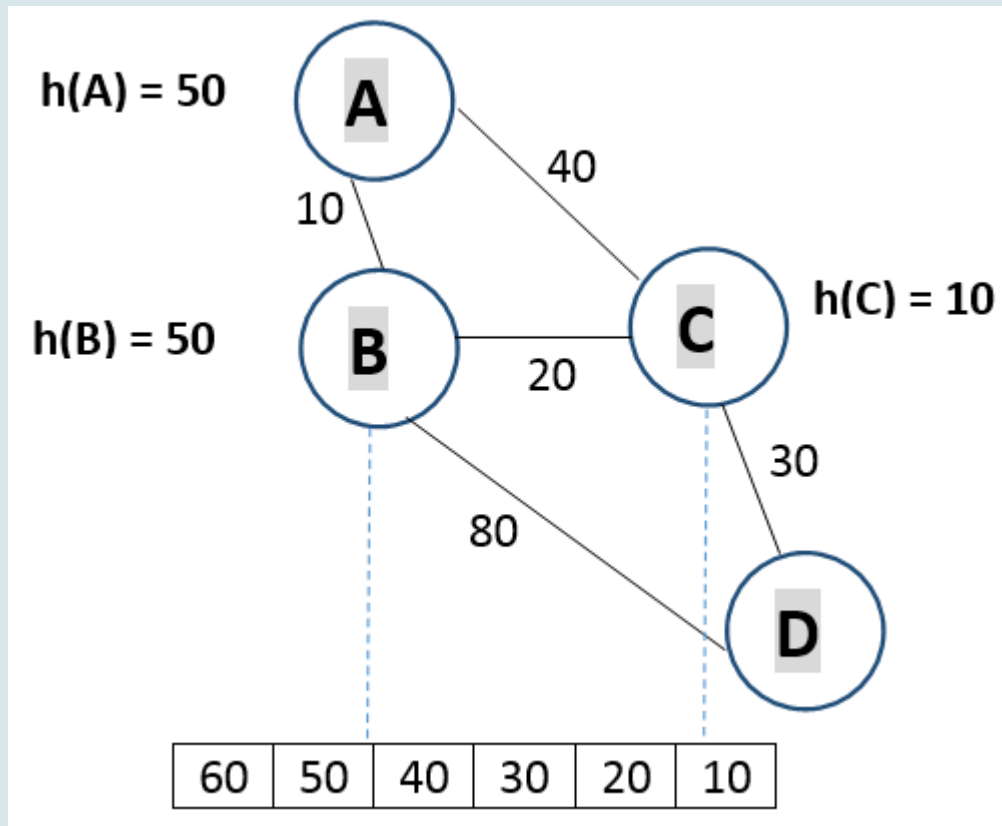
The heuristic given here is **admissible** because it does **not overestimate the true path cost from any given node**.

A:	$50 \leq (10+20+30)$	→ true
B:	$50 \leq (20+30)$	→ true
C:	$10 \leq 30$	→ true

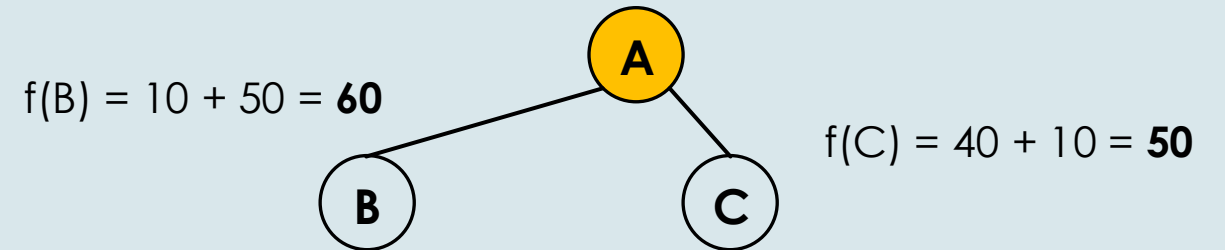
Evaluation function: $f(n) = g(n) + h(n)$

Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality



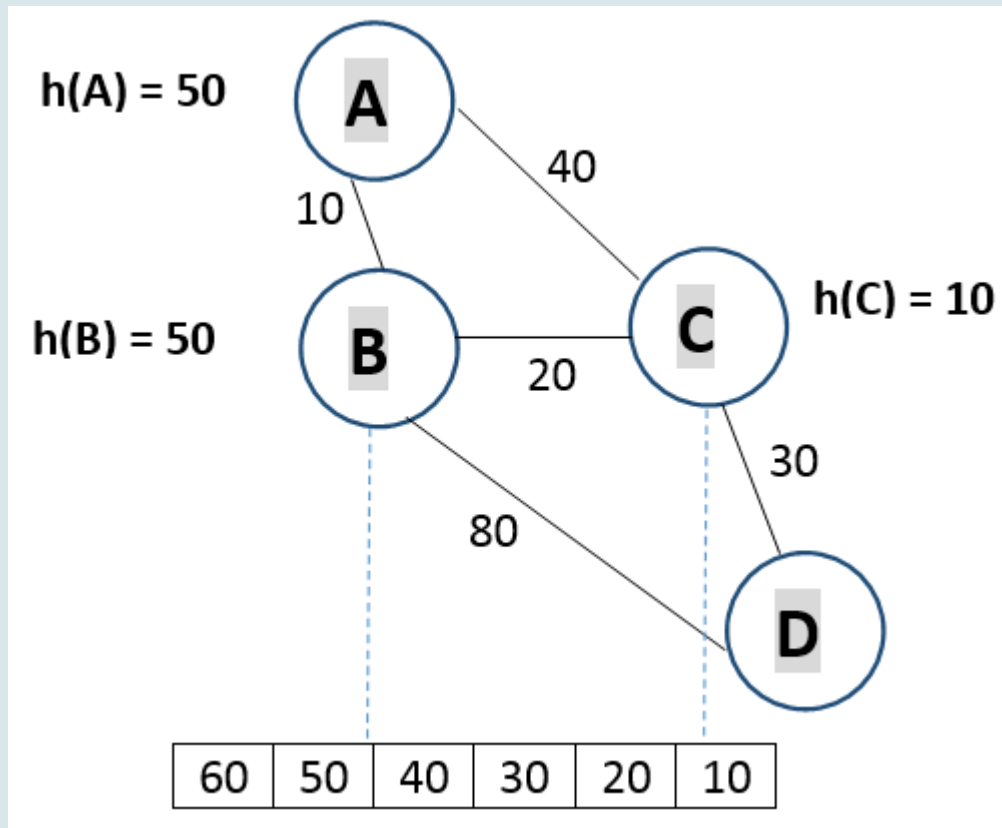
An admissible heuristic will ensure optimality when we use tree search.



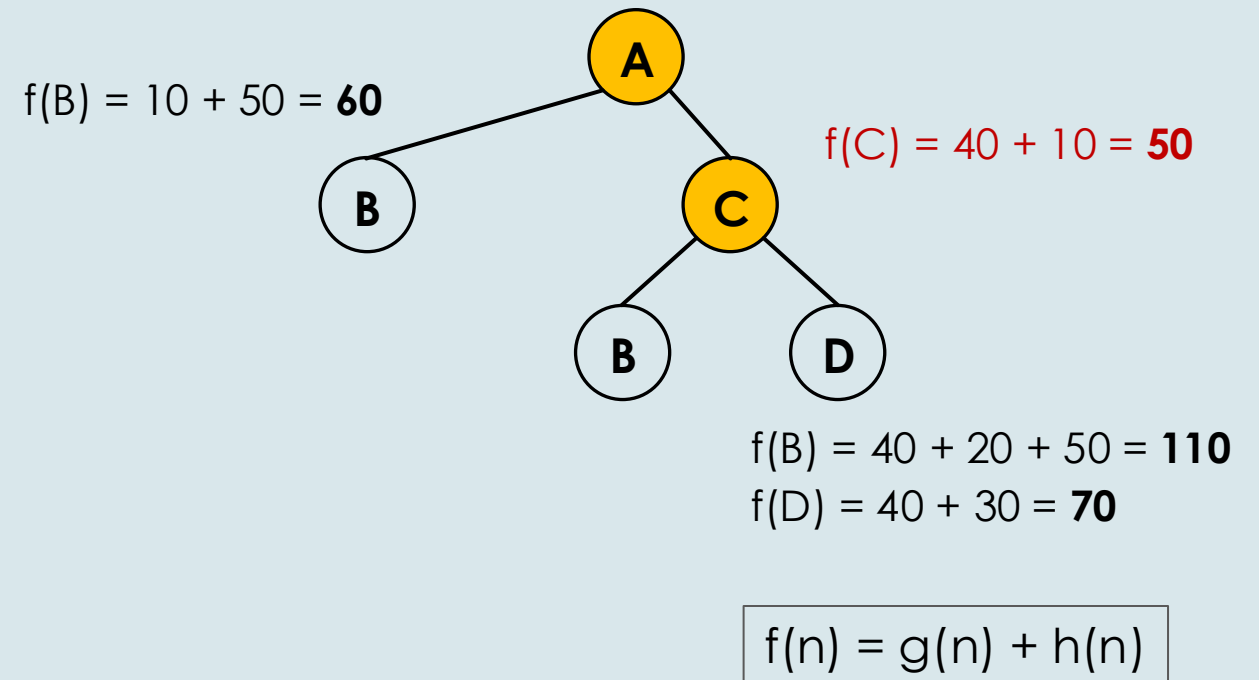
$$f(n) = g(n) + h(n)$$

Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality

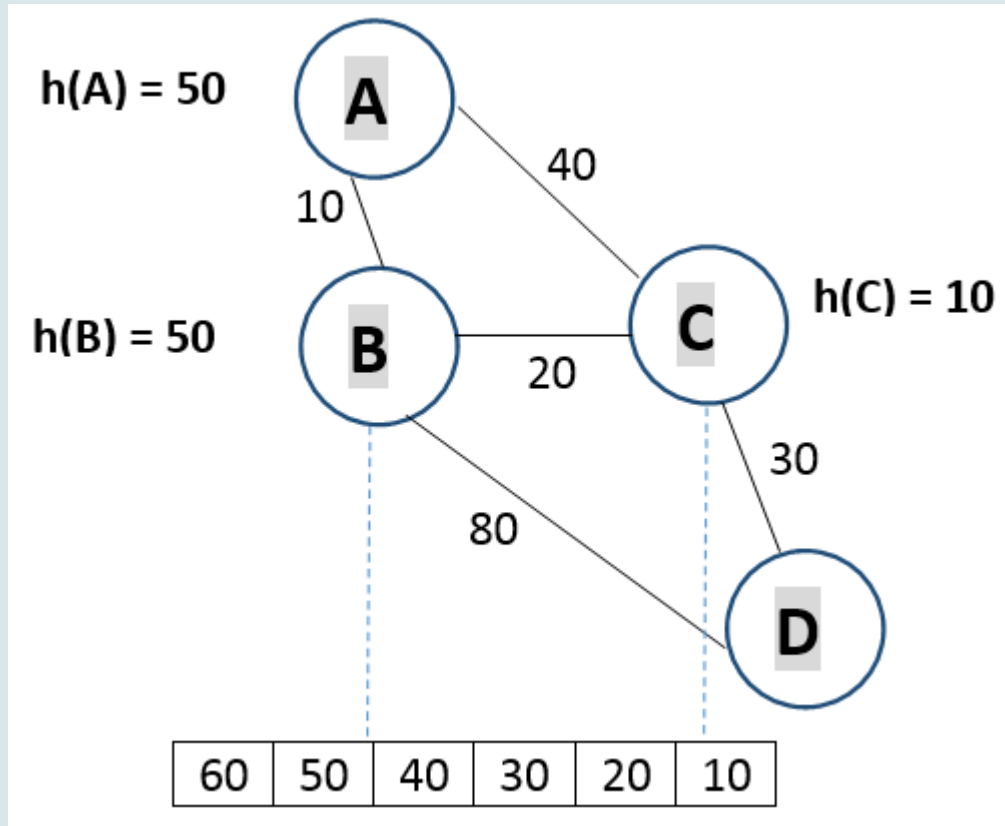


An admissible heuristic will ensure optimality when we use tree search.

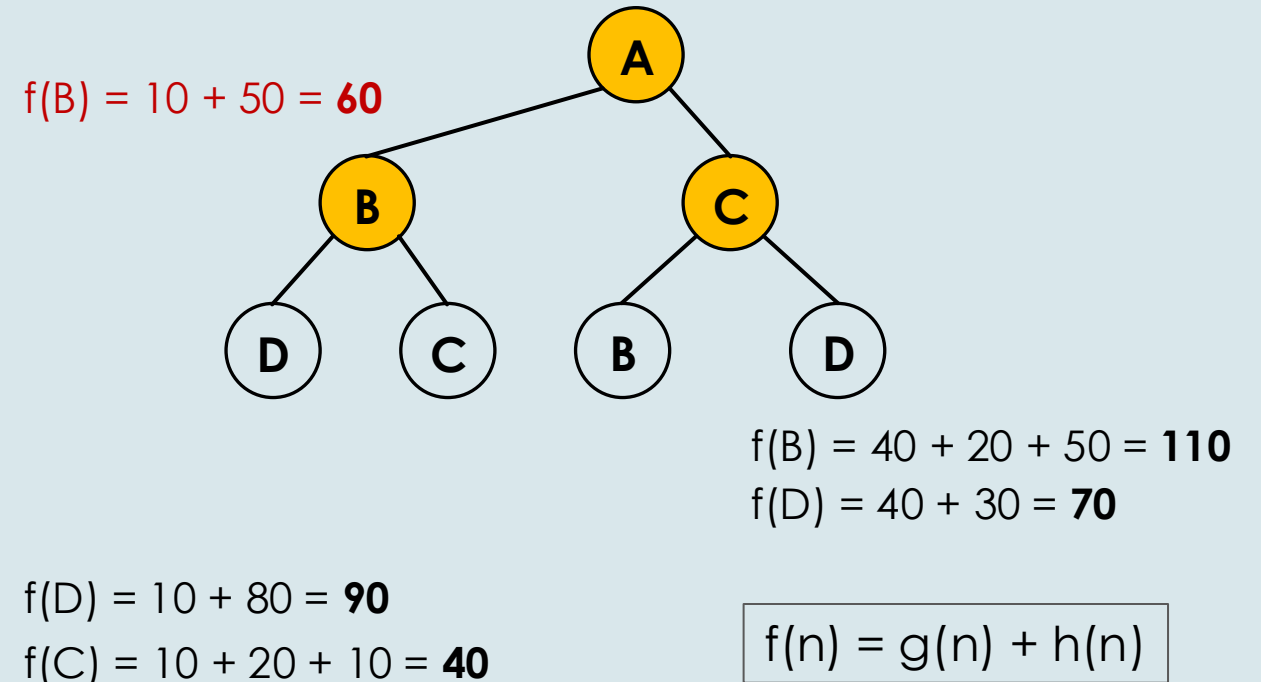


Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality

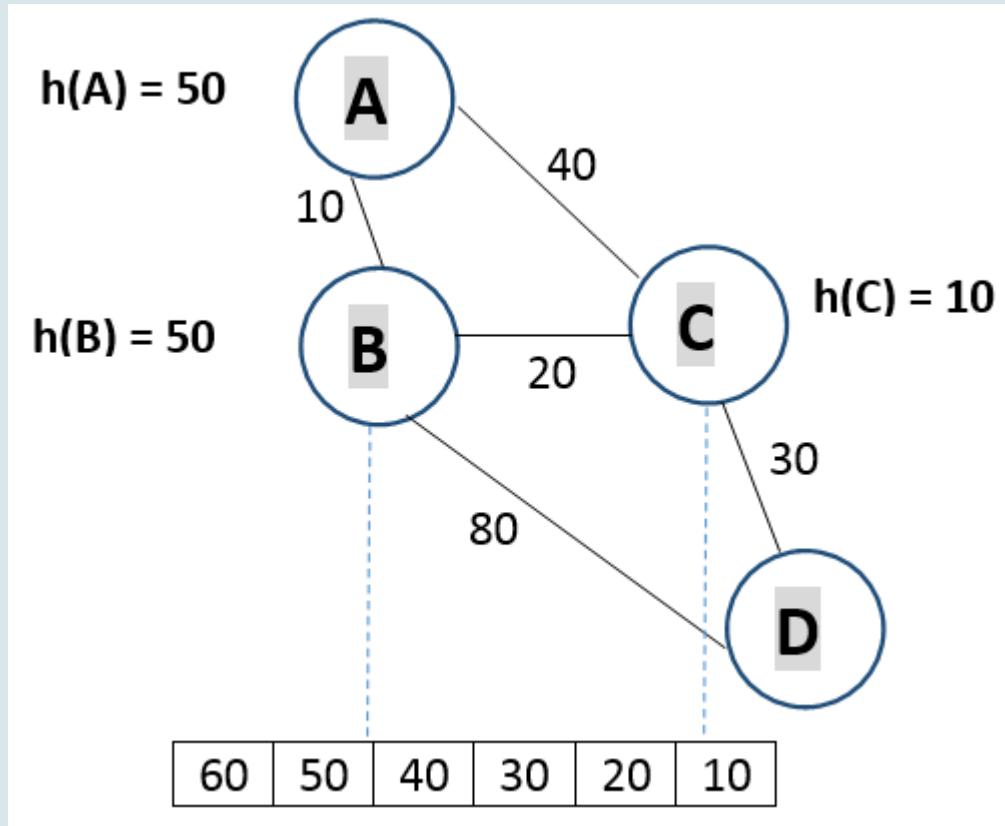


An admissible heuristic will ensure optimality when we use tree search.

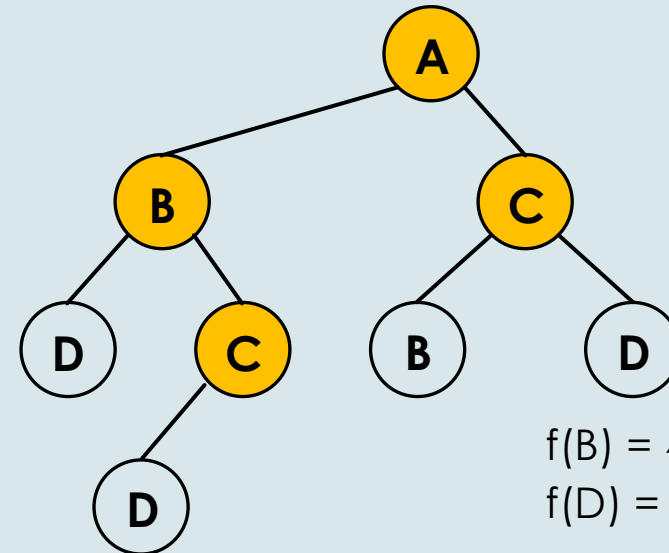


Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality



An admissible heuristic will ensure optimality when we use tree search.



$$f(B) = 40 + 20 + 50 = 110$$
$$f(D) = 40 + 30 = 70$$

$$f(D) = 10 + 80 = 90$$

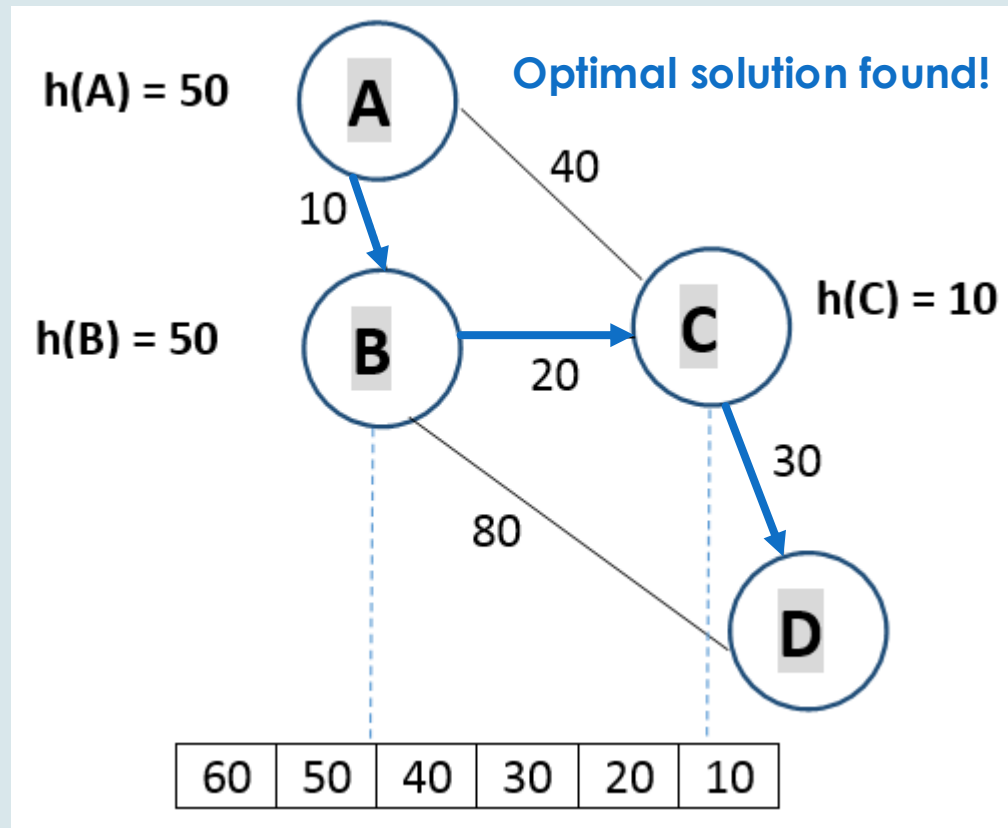
$$f(C) = 10 + 20 + 10 = 40$$

$$f(D) = 10 + 20 + 30 = 60$$

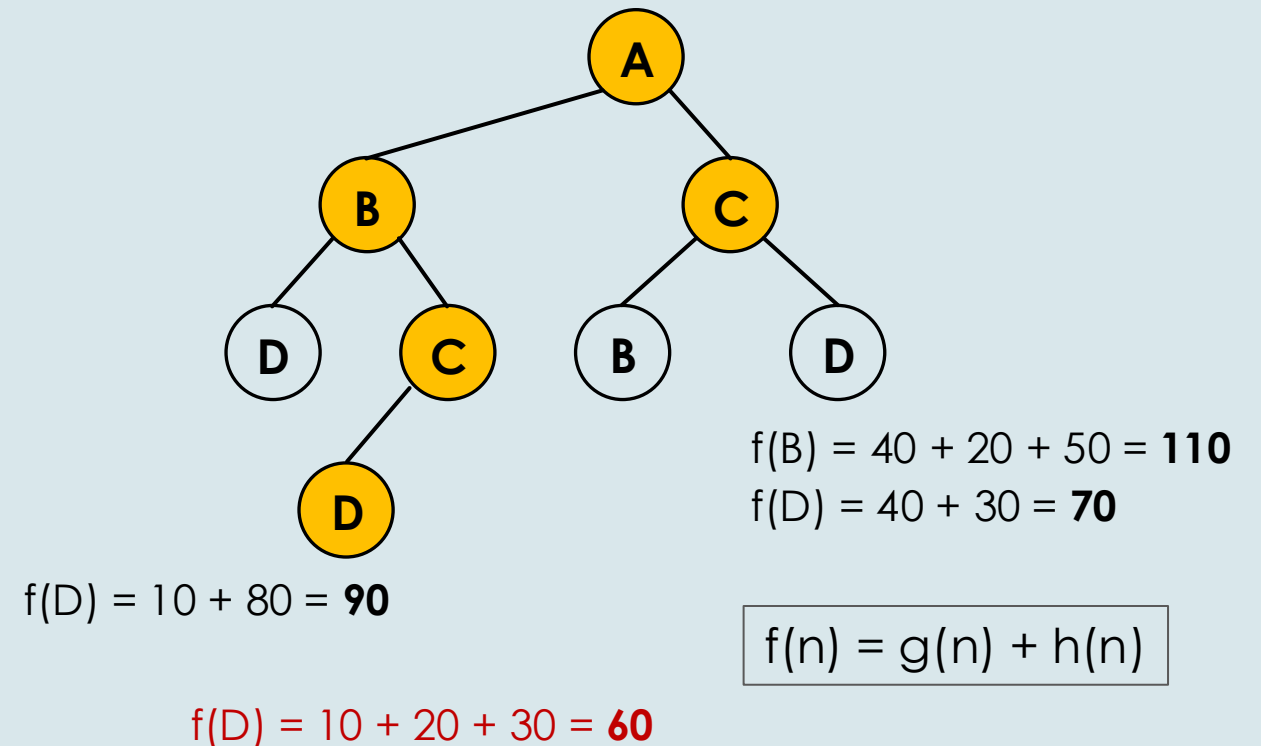
$$f(n) = g(n) + h(n)$$

Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality

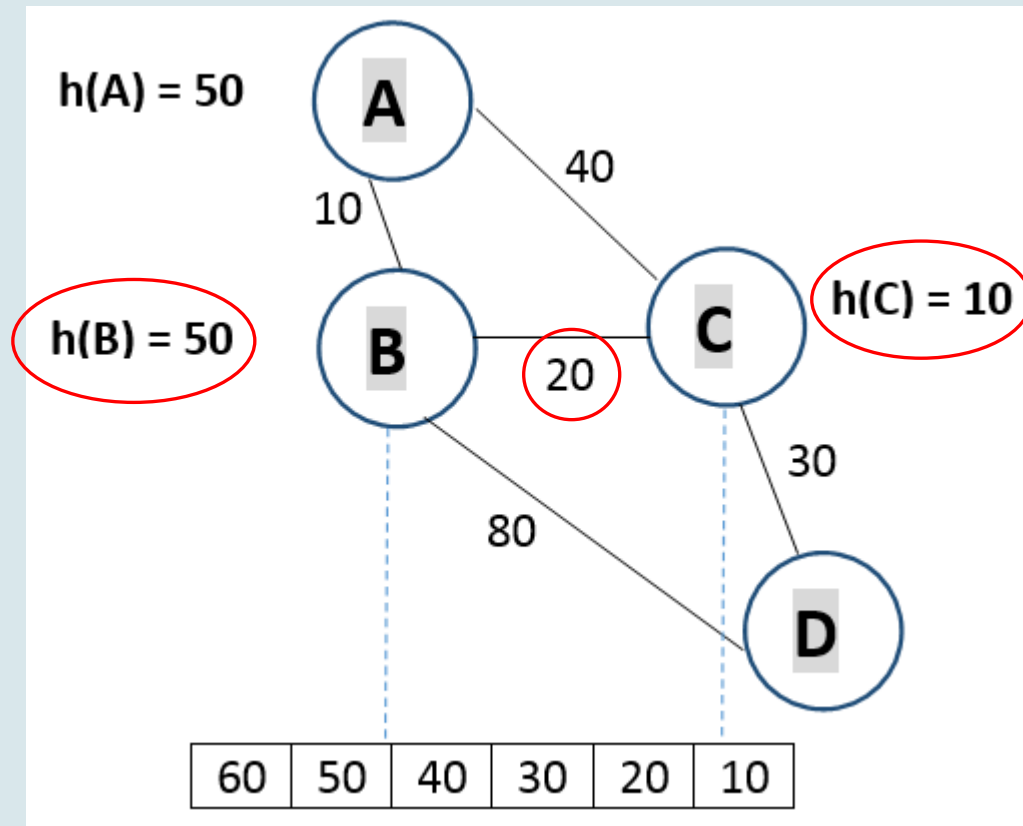


An admissible heuristic will ensure optimality when we use tree search.



Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality



A **consistent** heuristic will ensure optimality when we use **graph search**.

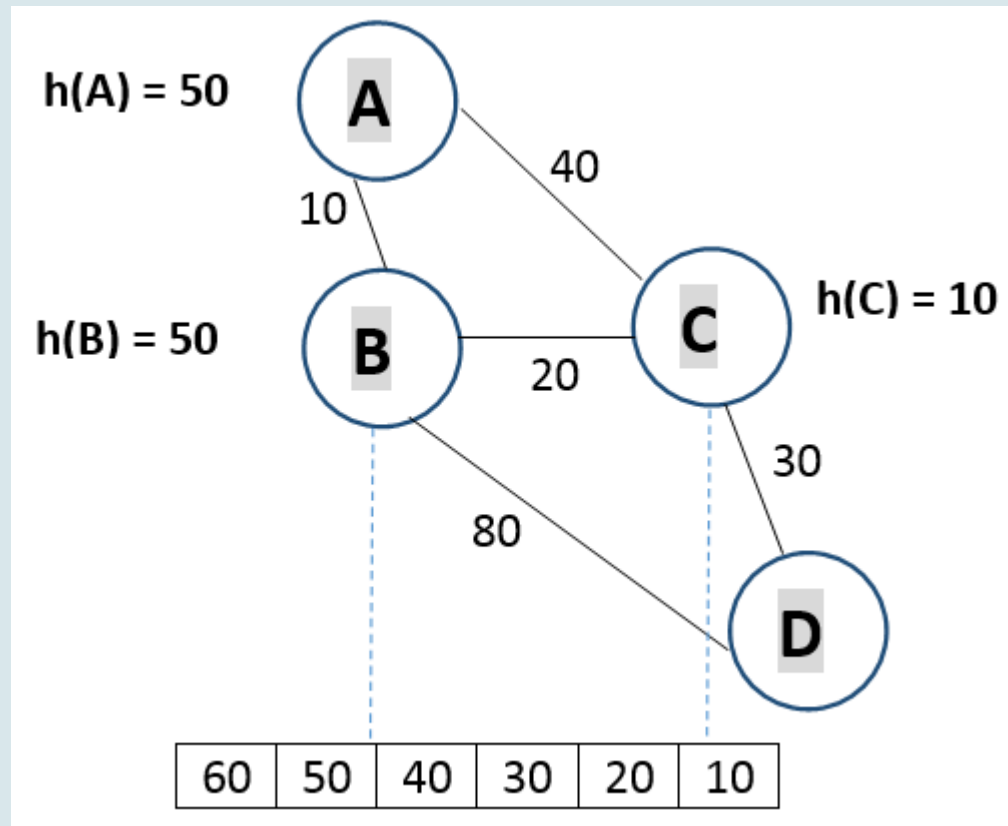
The heuristic given here is admissible because it does not overestimate the true path cost from any given node.

However, it is **not consistent**, because the heuristic **overestimates at node B** in terms of the **step cost from B to C plus the estimated cost from C to the goal**.

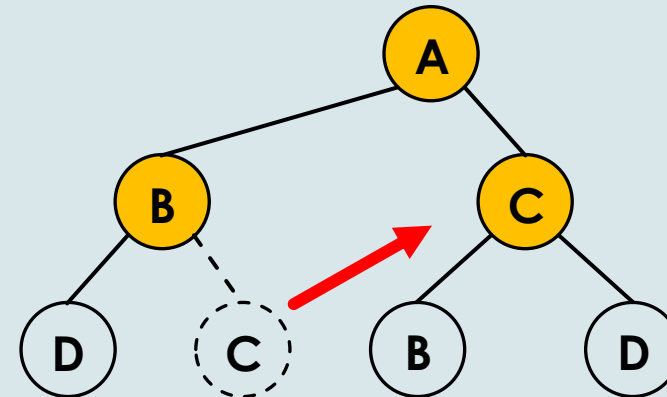
A: $50 \leq (40+10) \leq (10+50) \rightarrow \text{true}$
B: $50 \leq (20+10) \rightarrow \text{false!}$

Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality



In this example, we will miss the optimal solution using **graph search** because our heuristic is **not consistent**.



We will not discover the path from B to C because we visited C already.

$f(B) = 40 + 20 + 50 = 110$

$f(D) = 40 + 30 = 70$

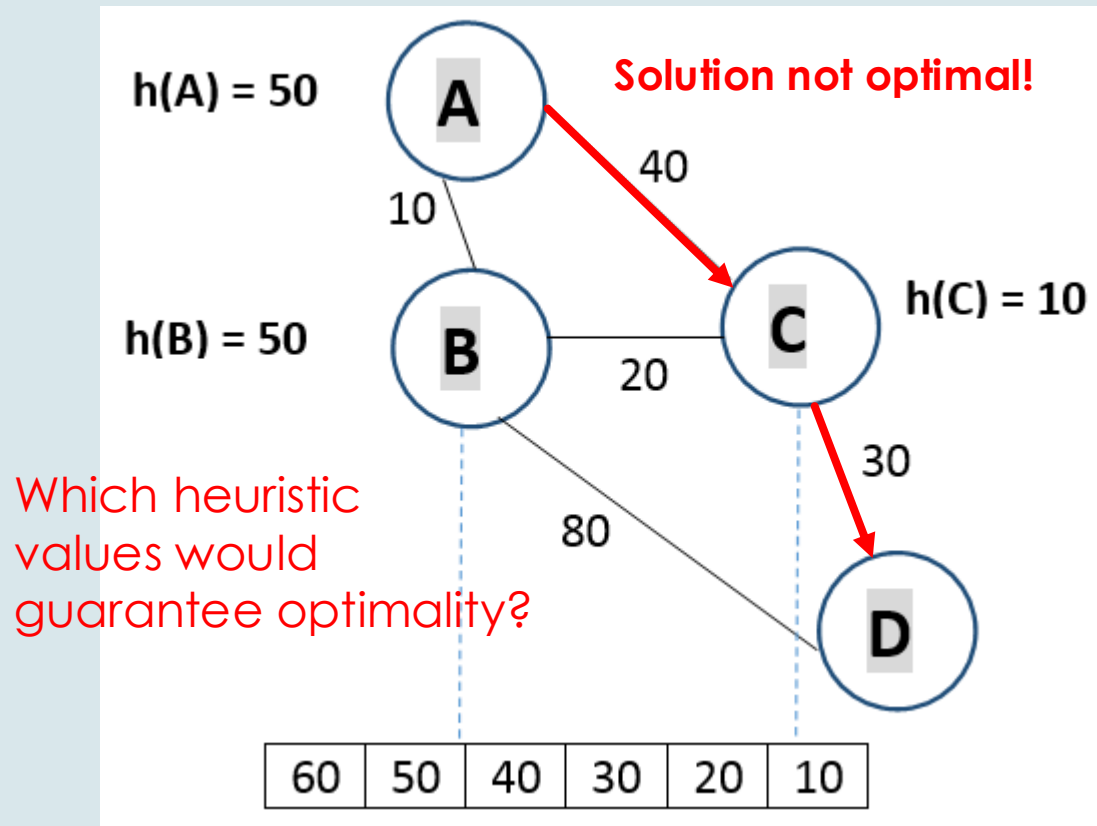
$$f(D) = 10 + 80 = 90$$

$$f(C) = 10 + 20 + 10 = 40$$

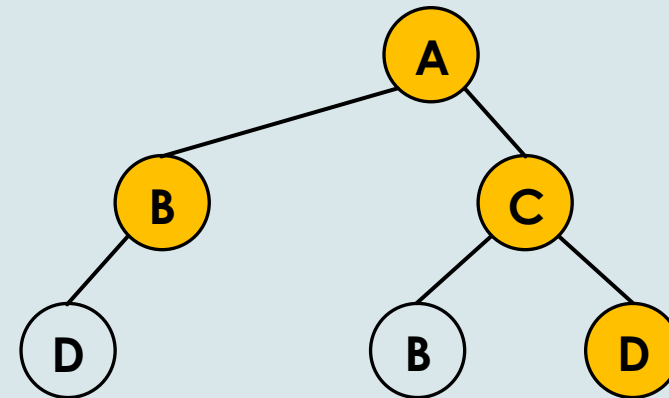
$$f(n) = g(n) + h(n)$$

Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality



In this example, we will miss the optimal solution using **graph search** because our heuristic is **not consistent**.



$$f(B) = 40 + 20 + 50 = 110$$

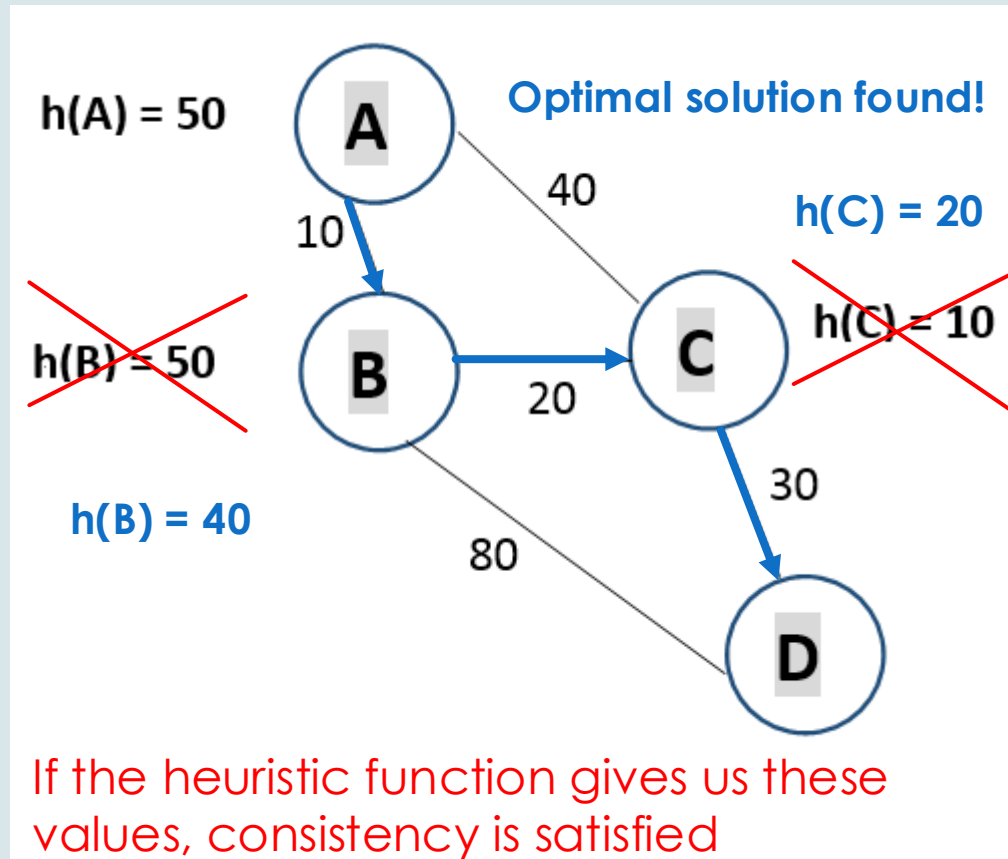
$$f(D) = 40 + 30 = 70$$

$$f(D) = 10 + 80 = 90$$

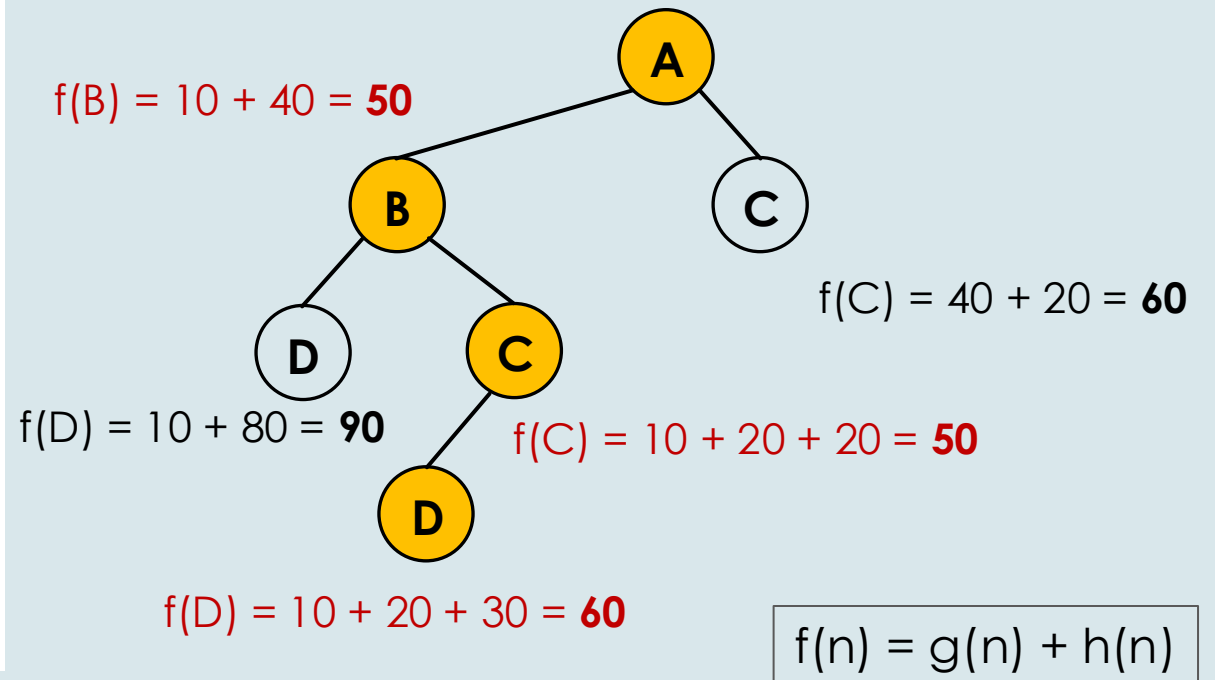
$$f(n) = g(n) + h(n)$$

Optimality in Heuristic Search

Admissibility & Consistency as Conditions for Optimality



A **consistent** heuristic will guarantee optimality when we use **graph search**.



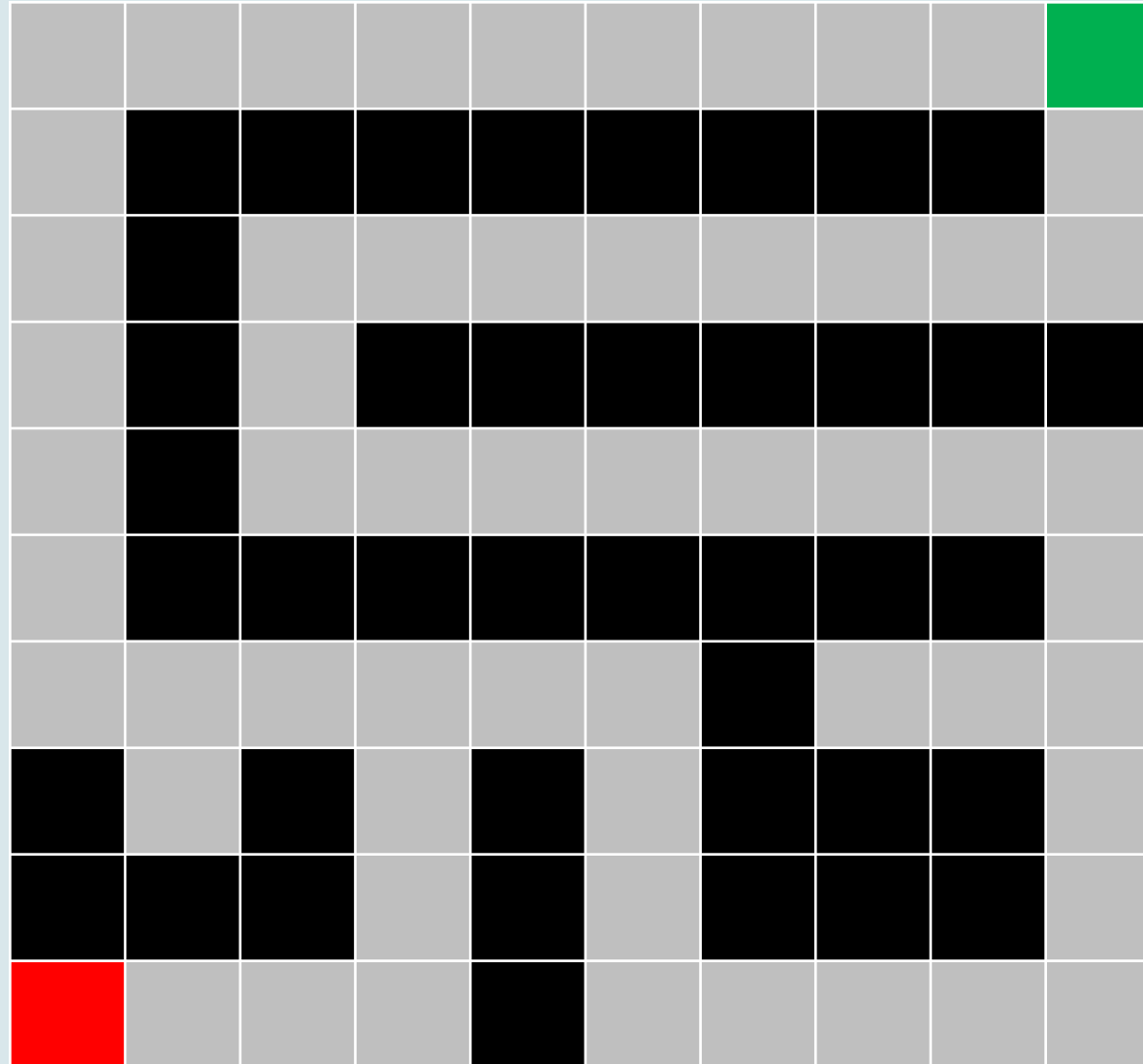
- The consistency requirement ensures there is no over-estimation at a **successor** node
- With graph search, there is the potential to ignore (i.e., not revisit) nodes throughout the search space

Why is checking successor nodes enough?

- The effect is to check that every h estimation is admissible, and that each is consistent with its successor counterpart
- The overall effect is to check that h estimations are consistent throughout the search space

Optimality in Heuristic Search

11



Optimality in Heuristic Search

12

9	8	7	6	5	4	3	2	1	
10									1
11		9	8	7	6	5	4	3	2
12		10							
13		11	10	9	8	7	6	5	4
14									5
15	14	13	12	11	10		8	7	6
	15		13		11				7
			14		12				8
	17	16	15		13	12	11	10	9



Optimality in Heuristic Search

13

9+15	8+16	7+17	6+18	5+19	4+20	3+21	2+22	1+23	
10+14									1+37
11+13		9+29	8+30	7+31	6+32	5+33	4+34	3+35	2+36
12+12		10+28							
13+11		11+27	10+26	9+25	8+23	7+23	6+22	5+21	4+20
14+10									5+19
15+9	14+8	13+7	12+6	11+7	10+8		8+20	7+19	6+18
	15+9		13+5		11+9				7+17
			14+4		12+10				8+16
	17+1	16+2	15+3		13+11	12+12	11+13	10+14	9+15